One way to test whether this kind of false sharing is a problem is to add huge blocks of padding between the data elements that can be concurrently accessed by different threads. For example, you can use

```
struct protected_data
{
    std::mutex m;
    char padding[65536];
    my_data data_to_protect;
};
```

65536 bytes is orders of magnitude larger than a cache line

to test the mutex contention issue or

```
struct my_data
{
    data_item1 d1;
    data_item2 d2;
    char padding[65536];
};
my_data some_array[256];
```

to test for false sharing of array data. If this improves the performance, you know that false sharing was a problem, and you can either leave the padding in or work to eliminate the false sharing in another way by rearranging the data accesses.

Of course, there's more than just the data access patterns to consider when designing for concurrency, so let's look at some of these additional considerations.

8.4. Additional considerations when designing for concurrency

So far in this chapter we've looked at ways of dividing work between threads, factors affecting performance, and how these factors affect your choice of data access patterns and data structures. There's more to designing code for concurrency than just that, though. You also need to consider things such as exception safety and scalability. Code is said to be *scalable* if the performance (whether in terms of reduced speed of execution or increased throughput) increases as more processing cores are added to